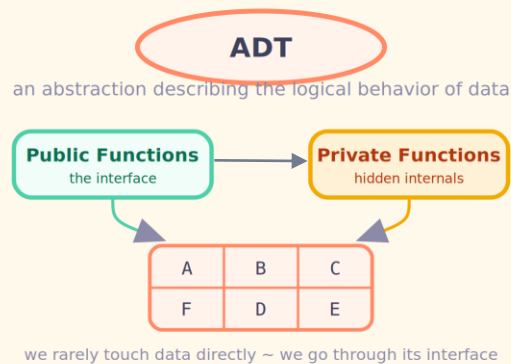


Arrays

1. Before Data Structures: what's an ADT?

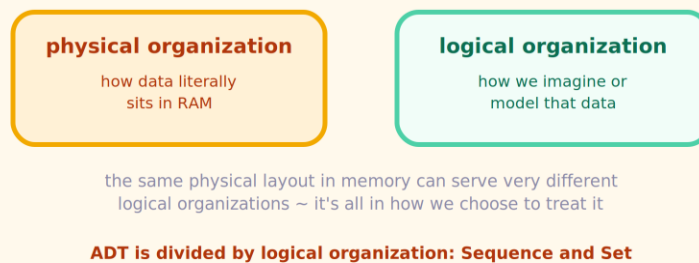
Before diving into any specific Data Structure, it helps to step back to one concept first: the ADT (Abstract Data Type).



When we handle data, we rarely interact with it directly — usually we go through some interface that controls how we're allowed to work with it. An ADT is exactly that: an abstraction, or a logical description, of data and how its pieces are meant to interact — regardless of how it actually sits underneath.

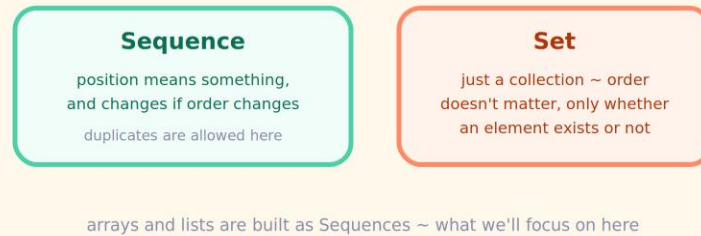
That interface is made up of Public Functions (what we're allowed to call from outside) and Private Functions (the internal mechanics hidden from us).

Physical vs. logical organization



~ the same physical memory layout can be treated completely differently depending on which logical organization we choose to impose on it.

An ADT is split up based on this logical organization — in other words, what we consider the data to actually represent.



Sequence: position carries meaning, and that meaning changes if the order changes. Duplicates are allowed.

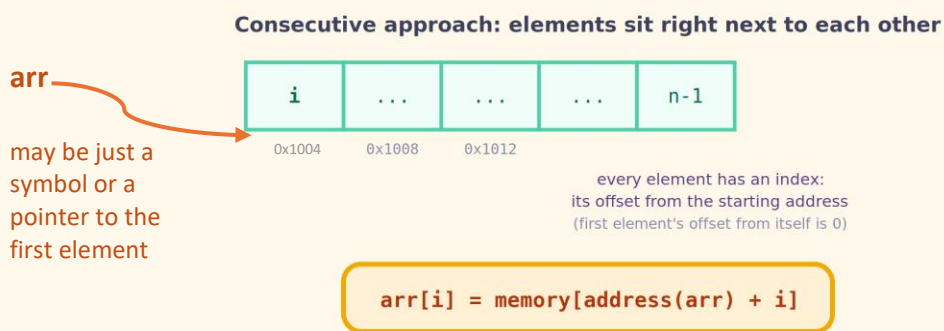
Set: just a collection of elements — what matters is only whether an element exists, not its position, and there's no duplication.

2. What about physical organization ?

Now that we've covered logical organization, what about physical organization — how data actually sits in RAM? We'll focus on the Sequence of Data, and there are 2 general approaches for building Data Structures physically:

- Consecutive — elements are placed directly next to each other in memory.
- Pointer-based — elements are scattered, each one linked to the next.

An array is the main example of the Consecutive approach — the simplest way to build a Sequence.



Each element has an index — which is really just its offset from the starting address of the array. The first element's offset from itself is mostly zero.

~ this formula is also why array access is $O(1)$: direct access, with no extra work needed to figure out where an element lives.

3. Time Complexity: what is Big O?

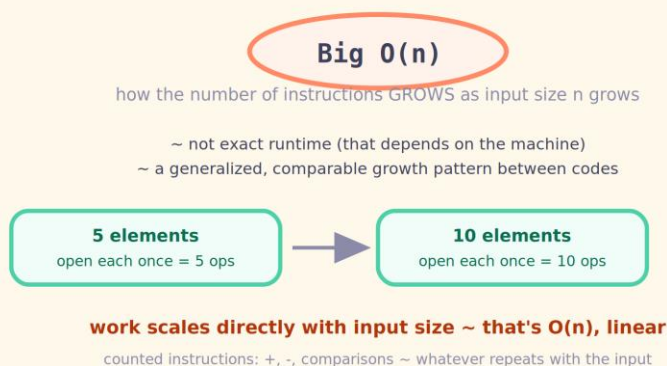
Growth means: given an input of a certain size, how much work does the algorithm actually need relative to that size? We're after a short, clean way to express how much time an algorithm needs, without listing every tiny detail of what it does.

So how do we actually express that growth? Looking at exact runtime isn't reliable — that depends heavily on the machine running the code. Can I fairly compare two codes just by timing them? Not really — maybe one machine is simply faster than the other. So timing alone isn't a trustworthy way to judge which one is actually better.

What we need instead is a way to compare the number of instructions between codes that stays general, regardless of which machine each one is running on.

Big O is exactly that: the number of instructions that scale with the input — specifically, whichever part grows the most as the input grows large, since that's the part that ends up dominating everything else.

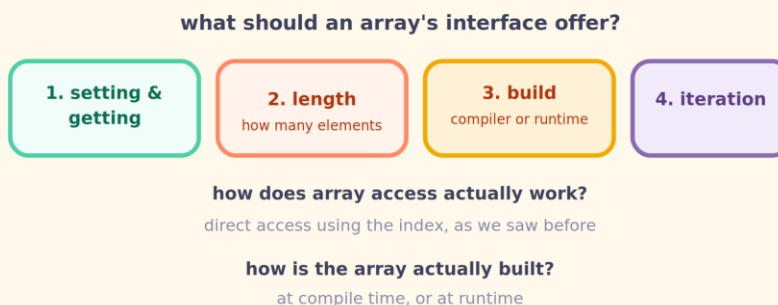
As a result, this kind of comparison stays consistent and independent of the machine environment, the code, or the language — and it stays meaningful even as things change.



~ What counts as an 'instruction' here is anything whose repetition depends on the input size — things like +, -, or comparisons. Or in the array example: how many times do we need to compare array x against array y?

4. Designing the array's interface

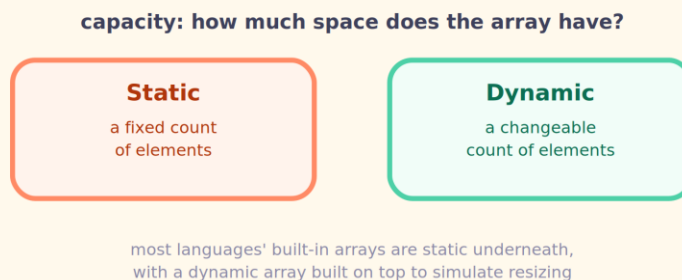
If we're the ones designing this Data Structure interface — what should it actually offer? What operations do we expect to need?



Access is direct, using the index, exactly as covered earlier. Building the array can happen either at compile time or at runtime, depending on the language/compiler.

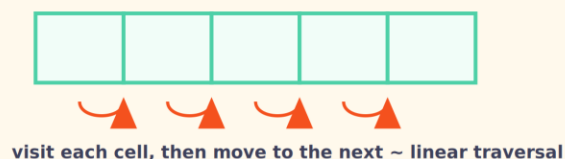
Static vs. dynamic capacity

Capacity means: how much space does the array actually have? This raises a question — are we dealing with a Static structure (a fixed element count) or do we want the freedom to resize?



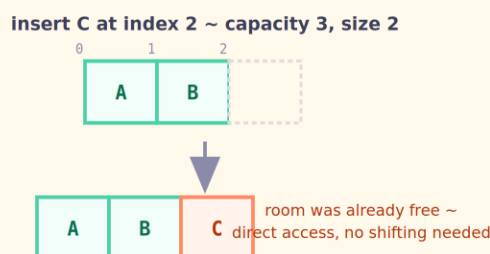
5. Iterating over an array

How do we iterate? Visit one memory cell, then move to the next, and so on — a linear traversal across the array.



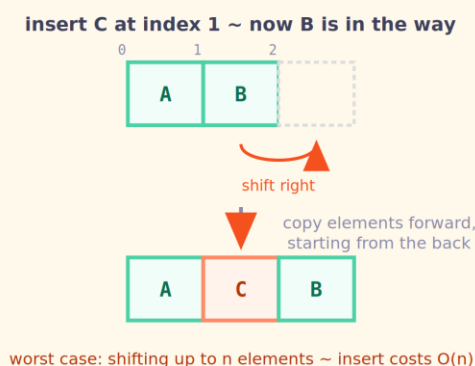
Inserting and Deletion

Say we want to insert C at index 2, in an array holding [A, B] with capacity 3 and size 2.



Since there was already free room at that position, direct access handles it — no shifting required.

But what if we want to add an element in the middle, where an existing element is in the way? Say we want to insert C at index 1:



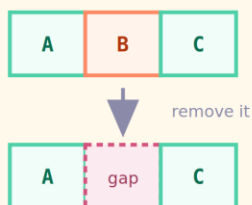
How do we actually do this? We shift every element after the insertion point one step to the right to open up space — this shifting happens by copying elements forward, starting from the back, so nothing gets overwritten before it's been copied.

~ If we add an element roughly in the middle, on average about half the array needs shifting. Overall, the cost stays proportional to the array size and depends on how packed the array is near where we're inserting — which gives it a growth rate of $O(n)$.

6. Deleting from an array

The same spirit applies here, in reverse: we delete an element, but we want access to stay direct, meaning the array shouldn't end up with a hole in the middle.

delete B ~ to preserve direct access from the front



a gap breaks the array shape ~ so we shift elements after it left, closing the gap ~ this also costs up to $O(n)$ in the worst case

But simply removing an element like this leaves a gap in the array shape. So, consequently, we need to shift the n elements that came after the gap, moving each one back by one position to close it.

7. Dynamic arrays: handling full capacity

What about when the array is already full and we still want to add something? Static capacity means we can't simply grow in place — there might not be any free space right after it in RAM.

So what do we do if we want a Dynamic array — one that can grow as needed? Capacity needs to grow, meaning we need a plan for resizing to a bigger size.

The idea: find a new spot in RAM with more room than the current array's capacity, copy every element from the old array into that new, bigger space, and once everything's copied, the old array is simply discarded, since its data now lives in the new one.

dynamic array: what happens when it's full?



the old array is discarded once every element is copied over

capacity + c

grows slowly ~ resizes often

capacity * c

grows fast ~ fewer, costlier resizes

averaged over many inserts, this still gives $O(1)$ amortized insert

So how should capacity grow — is it better to increase it by a constant amount (capacity + c), or multiply it (capacity * c)? This choice matters because of how often we end up resizing: growing too slowly means frequent, costly resizes.

~ Even though a resize itself costs $O(n)$ when it happens, it doesn't happen on every single insert — spread out over many inserts, the average cost per insert stays $O(1)$. This is what's called amortized $O(1)$: insert into a dynamic array averages out to constant time overall.